

برنامه کارآموزی آزمایشگاه رباتیک

تمرین هفته اول: طراحی یک سیستم ساده اجرای وظایف ربات

مقدمه

در سیستم‌های رباتیک، معمولاً وظایف مختلفی به صورت یک دنباله از عملیات به ربات داده می‌شوند. برای مثال، یک ربات ممکن است لازم باشد به یک ایستگاه حرکت کند، یک شیء را بردارد، به محل دیگری منتقل شود و آن را قرار دهد.

در چنین سیستمی لازم است بخشی از نرم‌افزار وظایف را مدیریت و به ترتیب اجرا کند، بخشی وظیفه اجرای عملیات روی ربات را بر عهده داشته باشد و بخشی نیز وضعیت فعلی ربات را در اختیار سایر اجزای سیستم قرار دهد.

در این تمرین، باید یک نمونه ساده از چنین سیستمی را با استفاده از ROS 2 و Python پیاده‌سازی کنید.

هدف تمرین، آشنایی عملی با ساختار یک سیستم ROS 2، نحوه ارتباط چند Node با یکدیگر و استفاده از Topic و Service در یک مسئله نزدیک به کاربردهای واقعی رباتیک است.

در این تمرین حرکت واقعی ربات یا کنترل سخت‌افزار مورد نیاز نیست. اجرای عملیات ربات به صورت ساده و شبیه‌سازی شده انجام خواهد شد.

سناریوی مسئله

فرض کنید یک ربات باید مجموعه‌ای از عملیات را به ترتیب مشخصی انجام دهد.
برای مثال:

MOVE_TO_WORKSTATION
PICK_OBJECT
MOVE_TO_SHELF
PLACE_OBJECT

یک Task Controller وظیفه دارد این عملیات را به ترتیب به ربات ارسال کند.
ربات پس از دریافت هر عملیات، آن را اجرا می‌کند. از آنجا که در این تمرین ربات واقعی در اختیار نداریم، اجرای هر عملیات فقط با گذشت یک مدت زمان مشخص شبیه‌سازی می‌شود.
پس از پایان هر عملیات، ربات باید به Task Controller اطلاع دهد که عملیات با موفقیت به پایان رسیده است. سپس Task Controller می‌تواند عملیات بعدی را ارسال کند.
همچنین باید بتوان وضعیت فعلی ربات را از طریق یک Node دیگر بررسی کرد.

معماری سیستم

سیستم باید حداقل شامل سه Node زیر باشد:

Task Controller:

این Node مسئول مدیریت صف عملیات و ارسال آنها به Robot Simulator است.

Task Controller باید یک Queue شامل مجموعه‌ای از عملیات داشته باشد.

برای مثال:

MOVE_TO_WORKSTATION
PICK_OBJECT
MOVE_TO_SHELF
PLACE_OBJECT

Task Controller باید عملیات‌ها را به ترتیب از Queue برداشته و برای Robot Simulator ارسال کند.

تا زمانی که عملیات فعلی به پایان نرسیده است، نباید عملیات بعدی ارسال شود.

پس از دریافت پیام پایان عملیات، Task Controller باید عملیات بعدی را ارسال کند.

Robot Simulator:

این Node نقش ربات را در این تمرین شبیه‌سازی می‌کند.

Robot Simulator باید بتواند عملیات‌های تعریف‌شده را دریافت و اجرا کند.

برای هر عملیات یک تابع جداگانه در نظر بگیرید. برای مثال، برای عملیات PICK_OBJECT تابعی با مسئولیت اجرای این عملیات داشته باشید.

در این تمرین، این توابع نیازی به کنترل واقعی ربات ندارند و تنها باید اجرای عملیات را شبیه‌سازی کنند.
هر عملیات باید مدت زمان مشخصی برای اجرا داشته باشد.
برای مثال:

MOVE_TO_WORKSTATION 5 seconds
PICK_OBJECT 3 seconds
MOVE_TO_SHELF 5 seconds
PLACE_OBJECT 2 seconds

پس از پایان مدت زمان مربوط به هر عملیات، Robot Simulator باید اعلام کند که عملیات با موفقیت انجام شده است.
Robot Simulator همچنین باید وضعیت فعلی خود را نگهداری کند.

:Task Monitor

این Node وظیفه بررسی وضعیت فعلی ربات را بر عهده دارد.
Task Monitor باید بتواند با درخواست کاربر، وضعیت فعلی Robot Simulator را دریافت و نمایش دهد.
حداقل اطلاعاتی که باید نمایش داده شوند:
وضعیت فعلی ربات
عملیات در حال اجرا
مدت زمان سپری‌شده از شروع عملیات
برای مثال:

Robot Status

Status: EXECUTING
Current Action: PICK_OBJECT
Elapsed Time: 1.7 seconds

در صورتی که ربات در حال انجام هیچ عملیاتی نباشد:

Robot Status

Status: IDLE
Current Action: NONE

ارتباط بین Nodeها

برای ارتباط بین اجزای سیستم از ROS 2 استفاده کنید.

سیستم باید حداقل شامل ارتباطات زیر باشد.

ارسال عملیات

Task Controller باید عملیات را برای Robot Simulator ارسال کند.

/task/action

این ارتباط باید با استفاده از Topic پیاده‌سازی شود.

اعلام پایان عملیات

Robot Simulator پس از پایان هر عملیات باید نتیجه را اعلام کند.

/action_done

Task Controller باید این Topic را دریافت کرده و پس از دریافت پیام، در صورت وجود عملیات بعدی، آن را ارسال کند.

دریافت وضعیت ربات

Task Monitor باید بتواند وضعیت Robot Simulator را درخواست کند.

/get_robot_status

این ارتباط باید با استفاده از Service پیاده‌سازی شود.

اطلاعات مربوط به وضعیت فعلی ربات باید در پاسخ Service قرار داشته باشند.

Robot Simulator باید حداقل وضعیت‌های زیر را پشتیبانی کند:

IDLE
EXECUTING
ERROR

در صورتی که Robot Simulator یک عملیات ناشناخته یا نامعتبر دریافت کند، باید وضعیت خود را به ERROR تغییر دهد و اجرای صف وظایف باید متوقف شود.

ساختار تحویل پروژه

کد پروژه را در یک ROS 2 package مناسب قرار دهید.

ساختار نهایی پروژه باید شامل کدهای مربوط به Node های سیستم و فایل های مورد نیاز برای build و اجرای package باشد.

همچنین در کنار کد، یک فایل README.md قرار دهید و موارد زیر را در آن توضیح دهید:

- معرفی کوتاه پروژه
- معماری سیستم
- Node های استفاده شده
- Topic ها و Service ها
- نحوه build کردن پروژه
- نحوه اجرای Node ها
- نحوه اجرای تست ها
- نمونه ای از خروجی اجرای سیستم
- مشکلات یا نکات مهمی که در طول پیاده سازی با آنها مواجه شدید

نکته پایانی

در این تمرین هدف اصلی، پیاده سازی یک سیستم پیچیده نیست. هدف این است که مفاهیم پایه ROS 2 را در قالب یک مسئله کوچک اما نزدیک به ساختار سیستم های واقعی رباتیک به کار بگیرید.

در مراحل بعدی، اجزای شبیه سازی شده این پروژه می توانند با اجزای واقعی ربات، سنسورها، کنترلرها و سیستم های تصمیم گیری جایگزین شوند. بنابراین در طراحی Node ها و interface های ارتباطی، سعی کنید ساختاری منظم و قابل توسعه ایجاد کنید.

